✱ NUMPY ✱

✱ What is Numpy ?

- Numpy is Scientific computing library for Python
- Support large number of data in the form of multi-dimensional array and matrix.
- Use for mathematic calculation like linear algebra, fourier transform, and random number capabilities
- Written in Python and C
- File format data .tsv (tab separated value)
  .csv (comma separated value)

Importing Numpy

```
import numpy as np
```

✱ Practical in Numpy

i] Open Jupyter Notebook (on default browser)
ii] Create new folder for specific practice.
iii] Create new file .ipynb (Interactive Python Notebook)
iv] Rename file
v] Need to do code line in cell
vi] Execute
- Shift + Enter - Execute and create new cell.
- Ctrl + Enter - Only for current cell execution
- Run Button - Same work like Shift + Enter

✱ Note:

- a - insert a cell above
- dd - delete a cell
- b - insert a cell below

* type()
  shows the type of data.
* .ndim
  shows the dimension of array
* .size
  shows value of element
* .shape
  shows rows & column value
  (first one is row & second one column)
* .dtype
  shows datatype with bit

#Program:
```
import numpy as np
arr_1d = np.array([1,2,3,4])
print(arr_1d)          (O/P: [1,2,3,4])
type(arr_1d)           (O/P: numpy.ndarray)
arr_1d.ndim            (O/P: 1)
arr_1d.size            (O/P: 4)
```

* Note:
You need to reexecute all cells if file is reopened.
There is shortcut for execute all cell click on kernel in menu bar and click on Restart and Run All
Kernel → Restart & Run All

* np.ones()
Create 1s matrix
- Created matrix is in the float
- You need to add rows & column value in tuple
ex: np.ones((3,4))

* Note:
If you forget about any function usage you can just click Shift + Tab in round bracket
(Press tab after '.' for methods)

* np.zeros()
Create 0s matrix.

* np.empty()
Create empty matrix but matrix is not empty. It includes random numbers.

* np.eye(5) :- Identity matrix.

Note:- for heading

Cell → Cell Type → Markdown

use '#' for heading

Numpy Functions

1] * .arange()

Create an array between the given value where start value is same but end value is n-1.

np.arange (1, 13, 2)

np.arange (start, End, Steps)

2] * linspace()

Create an array like arange() but third value is for creating that number of element. np.linspace (1, 13, 4)

3] reshape()

The function change the shape of any dimensional array into any dimensional array but only take care of value numbers.

If second parameter is negative it'll ignored(1)

np. array.reshape(dimension, row, column)

4] ravel()

function changes the shape of any dimensional array into one dimensional array

array.ravel()

5] flatten()

This function changes the shape of any dimensional into one dimensional array like ravel(). There are four orders

'C' – row major (C-style)

'F' – column major (Fortran-style)

'A' –

'K' –

6] transpose()

Function changes the row into column & column to row.

array.transpose() or

array.T (Not working)

Numpy Mathematical Operations

1] add()

Addition element to element

np.add(array)

OR array + array

2] Subtract()

Subtraction element to element

np.subtract(arr1, arr2)

OR arr1 - arr2

3] multiply()

Multiplication by element to element

np.multiply(arr1, arr2)

OR arr1 * arr2

4] divide()

Divide by element to element

np.divide(arr1, arr2)

OR arr1 / arr2

5] dot()

Divide by first row first column

arr1.dot(arr2)

OR arr1 @ arr2

6] max()

Maximum number in matrix

arr1.max() OR np.max(arr1)

max(axis = 0 or 1) 0-column, 1-row

7] argmax()

Index of maximum number in matrix

arr1.argmax() OR np.argmax(arr1)

8] min()

Minimum number in matrix

arr1.min() OR np.min(arr1)

min(axis = 0 or 1) 0-column, 1-row

9] argmin()

Index of minimum number in matrix

arr1.argmin() OR npargmin(arr1)

10] Sum()

Sum of all element

np.sum(arr1)

np.sum(arr1, axis=0) #0 represent column

np.sum(arr1, axis = 1) # 1 represent rows

11] mean()

Average of matrix

np.mean(arr1)

Return float value

12] sqrt()

Square root of every element

np.sqrt(arr1)

13] exp()

e^x where e is Euler's number = 2.718281

np.exp(arr1)

14] std()

Standard deviation (average square)

np.std(arr1) # gives one value

15] log()

Natural log

np.log(arr1)

16] log10()

np.log10(arr1)

Numpy Array Slicing

mx is the 1 to 100 numbers matrix.

```
mx[0,0]          # access one element
mx[0,0].ndim     # dimension of the element '0' means it's scalar
mx[1,0]          # first value is row & second is column
mx[0]            # access the indexed ~~item~~ row
mx[:,0]          # access the indexed column
                 : means whole row or column
mx[:,0:2]        # shows value in 2D
mx               # access whole matrix
mx[:], mx[::], mx[:,:]  # access whole matrix
```

.itemsize - size of item is 4

.dtype - shows int32 ; datatype of items 32=4*8

mx[0][1,2,3] - second bracket for multiple element

Numpy Array Concatenation and Split

1] .concatenate()

Combine two matrix but by default row wise or vertically.

Need tuple in this function

```
np.concatenate((arr1, arr2))      # row
np.concatenate((arr1, arr2), axis=1) # column
```

2] .vstack()

Vertically concatenation

3] .hstack()

Horizontal Concatenation

4] .split()

Split array/matrix

```
np.split(arr1,2) # 2 shows the split number
```

Value/output shows in list

By defaut split in row wise

```
np.split(arr1,2, axis=1) # colum wise
```

If d1 is array

```
np.split(d1, [1,3]) # index n-1
```

* Numpy Trigonometry function

Need to import matplotlib.pyplot

```
import matplotlib.pyplot as plt
```

1] np.sin(x), np.cos(), np.tan()

Shows the value

np.sin(np.radian(30))

2] plt.plot(x,sin, y,sin)

Create a graph with x & y axis

3] plt.show()

The important function to show the graph.

* Numpy Random Sampling

1] np.random.random()
- For random number in float
- You can add more values also
- For matrix value gives in tuple

2] * np.random.randint() (1, 100, (4,5))
- ~~first parameter is for how many values (integer)~~
- ~~Second Parameter sets limit~~
- Takes two parameter for in between range
- For matrix parameter need to gives in tuple

3] np.random.seed(2**32-1)
- Function store the random value
- In bracket is the value for limit. 4294967295
- You need to mention number in bracket like inder.

4] * np.random.rand()
- gives value between 0 to 1
- No need of tuple for matrix

5] * np.random.randn()
- Negative values also given by this function
- Uniformally distributed data (Bell Curve)

6] np.random.choice()
- Gives the random number in the mentioned array of bracket

7] np.random.permutation()
· Function changes the sequence of array element by randomly

* Numpy string functions

1] np.char.add(str1, str2)
- Add two strings

2] np.char.multiply(str1, 5)
- String multiply by the number in braces

3] np.char.lowercase(str1)
- Change string case to lower

4] np.char.upper(str1)
- Change string case to upper

5] np.char.title(str1)
- Uppercase the first letter of every word

6] np.char.center(str1, 60, fillchar="*")
- Center align the string in given number (60) & other blank space is filled with given character

7] np.char.split()
- This function split every word of string
- Store every string in list.

8] np.char.splitlines("\n")
- \n is used to split string in that position

9] np.char.join([:], [str1])
- Join every word in string with given character

10] np.char.replace(str1, "old_word", "new word")
- Replace the old word of string to new word

11] np.char.equal(str1, str2)
- This function check the every word of both string is similar or not
- If similar gives True otherwise false
- function is Case sensitive

12] np.char.count(str1, "letter/word")
- count letters how many times present in the string
- ~~Output shows the position of word starting from~~
- 0 means not present

13] np.char.find(str1, "word")
- This function find the given word/letter in string
- Output shows the position of word starting from
- This function is case sensitive
- -1 means not present the word in string

✱ PANDAS LIBRARY ✱

✱ What is Pandas ?

- Pandas is a powerful Python data analysis toolkit
- Open Source
- A fast & efficient Dataframe object for data manipulation.
- Reading & writing data structures & different formats: csv, tsv, txt, XML, JSON, ZIP, etc.

✱ Pandas Data Structures

- Series (1D)
- Dataframe – widely used (2D)
- Panel (3D)

✱ Pandas vs Numpy

- Numpy array is used for the implementation of Pandas data objects.

✱ How to Use Pandas Library

```
import pandas as pd
```

- `pd.__version__`
  for version of Pandas

* Pandas Series

1] pd.Series()

- Series is a datatype (structure) in Pandas
- S is capital
- Use this function to create series data (like list)

pd.Series([]) - creates an empty series

```
S1 = pd.Series([1,2,3,4], index = ['a','b','c','d'])
```

Use index for creating index in series

By default index is starting from 0.

- dtype can change the datatype in series
- Scalar values also created in series.
- Scalar means single

```
scalar_S = pd.Series(0.5)
```

- You can create dictionary

```
dict_S = pd.Series({'a':1,'b':2})
```

Accessing single value

```
S1[0]
```

2] max()

for maximum value in series

```
max(S1)
```

OR

```
S1.max()
```

3] min()

For finding minimum values

min(S1)

OR S1.min()

4] For write greater than value (>)

S1[S1>2]

5] Addition of two series (+)

S1+S2

- Addition of first to first, second to second
- It shows NaN for the value which is not index to index

= NaN - Not a Number

6] mean()

7] median()

8] mode()

- can get more than one values
- Maximum occured elements

9] var()

- variance

* Pandas DataFrame

Pandas DataFrame is two-dimensional, size-mutable, potentially heterogenous tabular data structure with labeled axes (rows & columns).

1] pd.DataFrame()
- Create an empty Data frame
- Column [], Index []

2] list will be converted into Data frame
- column & index automatically generated from 0.

3] Dictionary to Data frame
- Key will be converted to column

4] MultiDictionary also managable in Data frame
- NaN value added instead of blank value.

5] Series in Dictionary to Data frame
- Values of dictionary is the series

df.loc[100] ⟶ gives values related to 100 with all columns also

* CSV file in Pandas

- Comma Separated Value

Advantages of CSV file

- Universal
- Easy to Understand
- Quick to Create

* pd.read_csv()

- for reading csv file
- use double slash(\\) in path of file

'C:\\Users\\Kambl\\OneDrive\\student_result.csv'

- Or use forward slash '/'

* os.getcwd()

- get current working directory

1] df.column

- df is the variable name, which is Dataframe.
- This function gives all columns in table.

2] nrows = 1

- Gives 1 rows of table

df = pd.read_csv('file_path', nrows = 1)

~~3]~~

3] usecols = [0]

- Gives first column as output
- You need to declare column number

usecols = [2, 4, 7]

4] skiprows = [2] / 2

- [2] skips index = 2 row
- 2 skips first two rows
- [1,2,3] skips index = 1,2,3 rows

5] index_col = "Name"

- Makes 'Name' column as an index

6] header

- header = 1 makes ~~first~~ second rows as a column name.
- header = None makes 0 to n as a column name.

7] prefix

- prefix = 'value' makes value 0,...to valuen as a column name. but you need to mention header = None
-

8] names

- names = ['value'] makes value as a column name.
- You need to mention no. of columns = no. of value. for better visualization.

9] df.head() (df is variable stored read_csv file)

- blank value of head() print first 5 rows
- If you give parameter then print row no. of parameter

10] df.tail()

- print last five rows without giving any parameter
- If parameter is given then print rows no of parameter

11] dtype = {'ID': 'float64'}

- change datatype of ID into float64

12] true_values = ['Yes']

- change Yes into True

13] false_values = ['NO']

- Change No into false.

* Handling Missing Value

1] na_values = 'not value'

- change not value / given string into NaN
- pd.read_csv('file_path', na_values = 'not Available')
- Use list for more than on string values
- na_values = {'companies': 'novalue'}

Use dictionary for spe~~cfic~~ change NaN in specified colum

2] keep_default_na = False

- By default some values (#N/A, N/A, n/a) value change into NaN
- for keep that by default na values.
- Keep the data frame as it is.

3] na_filter = False

- na_filter decides to check or not check null values
- false means don't check null value.
- This code skips bg activity of checking null value which means performance boosting

4] df.isnull()

- Check null values if value is null gives True otherwise false

5] df.isnull().sum()

- sum of null value in front of column
- df.isnull().sum().sum print all sum of null values.

6] df.notnull()

- If value is not null print True, if null print false.

7] df.dropna()

- By default skips/drop rows of null values

8] df.dropna(axis = 1)

- 1 means column which means drop column if null value present.
- 0 means row
- By default 0 is the value

9] df.dropna(how='any')
- any drops if 1 value of null also present
- all drops if all row/column value null present.

10] df.dropna(thresh=2)
- Need minimum equal to thresh parameter not null values present to keep the row
- If not null values are less than thresh parameter drop that row.

11] df.dropna(subset=['column_name'])
- Drop row of null value in selected column present
- Need mention of column name in list.

12] df.dropna(inplace=True)
- Drop row if null value present.

* Fillna() in Pandas
- fillna fills the NaN values with given values

1] df.fillna(0)
- fills 0 in all NaN values in Dataframe.

2] df.fillna({'foldername':'value'})
- Nan values changed by the specified value for that specified folder.
- You can modify all folder's NaN value by different values.

3] df.fillna(method = 'ffill')
- ffill method of fillna function changes NaN value by above cell values.

4] df.fillna(method = 'bfill')
- bfill method changes NaN values by below cell values

5] df.fillna(method = 'pad')
- pad method works likes ffill method

6] df.fillna(axis = 1, any_filling_method)
- Any filling method work by axis = 1
- If axis = 1 & method = 'ffill' works, change NaN value by previous column cell value.
- This parameter will only work with other filling parameter, means it can't work alone

7] df.fillna(0, limit = value)
- This parameter changes NaN values with specified values which is mentioned in fillna but limit of changing values will be given in limit
- This parameter can't work alone.

8] df.fillna(5, inplace = True)
- This parameter changes the real Dataframe.
- This can't create another Df.

Pandas replace function

- Values to the dataframe are replaced with other values dynamically.

1] `df.replace(to_replace='old_value', value='new_value')`

- to_replace parameter needs old value.
- value parameter needs new value.
- No errors show if old value is not present in DF.

2] `df.replace([list], 0)`

- You can replace more than one values by one new value, but add all values in list [].
- No errors shows if old value is not present in DF.
- Shows value error if no. of old value not equal to new value in list

3] `df.replace({'foldername':'old_value'}, 'new_value')`

- You can replace old value from a specific folder to new value by using dictionary.
- No error shows if old value not present

4] `df.replace('[A-Za-z]', 0, regex=True)`

- Changes all Capital A to Z, & small a to z to 0.
- regex = True is mandatory.

5] `df.replace('old_value', method='ffill')`

- Replace old value by method ffill which is by default ~~by~~ row. means replaceable value is above cell.

6] df.replace('old-value', method='bfill')

- Replace old_value by method bfill but axis is not defined so by default it's row, means below cell is new value.

7] df.replace('old-value', 'new-value', inplace=True)

- inplace=True changes the original DF.

Pandas interpolate function

- Pandas interpolate function is basically used to fill NaN values in Dataframe or Series
- Very powerful function
- Uses various interpolation techniques.

1] df.interpolate()
- This function fill all NaN integer values automatically by average.

2] df.interpolate(method='linear')
- Work like blank interpolate function means by default method is linear

3] df.interpolate(method='time')
- for this method the date values format needs to be $ in timestamp.
- You can use below code for no error or use time method

```
df = pd.read_csv('file_path', parse_dates=['Date'])
```

- You can change the format of specified column.

4] df.interpolate(method='nearest')
- fills nearest value automatically

5] df.interpolate(method='polynomial', order=1)
- order is mandatory for polynomial method.

Pandas loc function & iloc function

1] df.loc[0]

- You can use loc for accessing rows & columns by label or boolean array.
- Only by label
- 0 is label for first row so first rows will be displayed

2] df.loc[[0,3]]

- If you want to access more than one rows, you need to mention labels of rows in list.

3] df.loc[4,'Class']

- If you want to access one value, need to declare first parameter - row label, second parameter column index.
- 0:3 also usable for row labeling.

4] df.loc[[list_of_boolean_values]]

- You can access rows by using boolean values only thing is no.of rows equal to no.of boolean values.

5] df.loc[df['column_name']<11]['column2_name']

- You can access limit value in specified column
- Column2_name is the displayed value.
- If column2_name is not declared then displays whole row of true condition.

6] df.iloc[0]

- .iloc df can access values (rows by only index
- Shows error if label mentioned.
- Use list/[] for better visualization.

7] df.iloc[[0,3]]

- Displays mentioned index rows.

8] df.iloc[:, 0]

- first index for row & second for column.
- It works like list.

9] df.iloc[[boolean_values]]

- You can access rows by boolean values
- no. of rows equal to no. of boolean values.

* Pandas groupby function

- Pandas groupby function is used to split data into groups based on some criteria.
- Any groupby operation involves one of the following operations on the original object:
  - Splitting the object
  - Applying a function
  - Combining the results

1] `gr1=df.groupby(by='section')`
`gr1`
- Create a group by column_name is 'section'
- In which same values in section merged.

2] `gr1.groups`
- Print all created group in given column_name
- row indexes are shown in created group.

3] `df.groupby(['column_1', 'column_2']).groups`
- Create groups by more than one column name.
- Uses list for column names
- groups for print all group values.

4] `dict(list(gr1))`
- Dictionary is the suitable for best visualization
- List also mandatory.

5] gr2 = df.groupby('column_name').get_group(group_value)
- gr2
- Print all values under group_values. which is created under column_name.

6] gr1.sum()
- gr1 is already created group
- sum() is used to summation of all group values in group in one value but in diff. column.

7] gr1.mean()
- mean() is used to calculate mean value.
- Other working is same like sum.

8] gr1.agg(['sum', 'mean', 'max'])
- aggregate function used for list of more than one inbuit function like sum, max etc.

* Pandas merge function

- Pandas merge connects columns or indexes in DataFrame based on one or more keys.

1] pd.merge(df1, df2)

- Need one column name to be same in both DF.
- Then merge the same name column & other columns as it is.
- Also need column values to be same.
- Only same values will be printed.

2] pd.merge(df1, df2, on='ID')

- Merge given on column name.
- ~~By default~~

3] pd.merge(df1, df2, on='ID', how='left')

- parameter of how are left, right, inner, outer
- inner ⇒ By default, print same value only in key column
- outer - print all values in key column of both DF
- left - print all values in key column of left DF
- right - print all values in key column of right DF

Note :- Print DF by rows so, if blank value fills with ~~Nan~~

4] pd.merge(df1, df2, on='ID', how='outer', indicator=True)

- Indicator takes boolean parameter
- Create new column named '_merge' values are prints presence of DF in row

Pandas concat function

- Pandas provides various facilities for easily combining together Series, DataFrames and Panel objects.

1] `pd.concat([Series1, Series2])`
- Use list for any types of parameter
- concatination will happen row wise (axis=0)
- By default axis is 0.
- By default indexing of both series stay as it is.
- Here, Series1, Series2 is the Series type parameter

2] `pd.concat([df1, df2])`
- Here, df1, df2 is DataFrame

3] `pd.concat([df1, df2], axis=1)`
- axis=1 gives result of concatination but in column wise; by default value is 0 means row wise.

4] `pd.concat([df1, df2], ignore_index=True)`
- By default ignore_index is false means when the DF is created the indexing will remain as it is while concatinate.
- ignore_index = True indexing of concatinated result starts from 0.

5] pd.concat([df1, df3], axis=1, join='inner')

- By default join value is outer means print the DF of concat with NaN values if both DF are not equivalent values.
- inner value prints only equivalent DF

6] pd.concat([df1, df3], keys=['first df', 'second df'])

- You can gives names for DF by using keys parameter

* Pandas join Function

- DataFrame join is a convenient method for combining the columns of two potentially differently-indexed

1] df1.join(df2)

- Need only indexing same
- If any column_name is same in both DF shows ValueError
- Join both DF by index not by column_name
- If no of rows in both DF are different then print DF by first DF.

2] df1.join(df4, how='left')

- If no of rows in both DF are unequal then handle result by using how parameter
- left - result shown by left DF
- right - result depends on right DF
- inner - result shown, only equivalent rows
- outer - shows all value of both DF by indexing & NaN values added in blank value.

3] df1.join(df5, rsuffix='_1')

- If column_name are same in both DF, rsuffix changes the name of right DF.
- Adds given value in right DF's same column
- You can also use lsuffix.

* What is Pandas Pivot Table ?

- The levels in the pivot table will be stored in multiIndex objects (Hierarchical indexes) on the index and columns of the result DF.

1] df.pivot_table(index='Genre')

- Genre is the column_name & after command it will be an index for DF.
- Average the values in all table but index format.

2] df.pivot_table(index='Genre', columns='Year')

- Column labeling also happen with column param.
- This also work like index but on column.

3] df.pivot_table(index='Genre', columns='Year', aggfunc='count')

- Count values in aggfunc can count how many values are present in that rows_name & column_name.
- max - maximum value
- Sum - Summation of values present in that rows_name & column_name.

4] df.pivot_table(index, columns, aggfunc='~~count~~ sum', margins=True)

- margin & aggfunc='sum' create new table & row & put all addition/summation value in it.

Pandas melt Function

- Pandas melt function is used to transform or reshape data

1] `pd.melt(df)`

- Create or change the shape of DF into index, variable & value format
- index - 0 to n
- variable - column_name
- value - inner data or cell data

2] `pd.melt(df, id_vars = 'Genre')`

- id_vars create one more column in it & arranges all data by it

3] `pd.melt(df, id_vars = 'Genre', value_vars = 'Ratings %')`

- shows only value under value_vars variable

4] `pd.melt(df, id_vars = 'Genre', var_name = 'Category')`

- var_name changes by defaut variable label to given argument

* Pandas DatetimeIndex functionality

1] `df = pd.read_csv('File_path', parse_dates=['column_name'])`
- Changes that columns datatype into datetime.
- Function's parse_dates parameter needs to use while importing file.

2] `df.dtypes`
- Shows all columns datatype

3] `type(df.Date[0])`
- Shows Date column's 0 indexed values type.

* MATPLOTLIB *

Q. What is matplotlib ?

- Matplotlib is 2D & 3D plotting python library.
- It produce high quality graph.
- It also support animation.

* History of matplotlib

- American neurobiologist
- John D. Hunter (1968-2012)
- Written in Python
- Initial released in 2003.

* Matplotlib Graphs

- Histogram
- Bar Charts
- Scatterplots
- Area Plot
- Pie Plot
- Error Charts
- Power Spectra

* Importing Matplotlib

1] from matplotlib import pyplot as plt

2] import matplotlib.pyplot as plt

Line Plot

1] plt.plot(x_axis, y_axis)

- plt.plot only plot a graph not shows it
- x_axis - you can give list in it
- y_axis - you can give list in it

2] plt.show()

- plt.show() shows the graph.
- In jupyter notebook it is not mandatory

3] plt.xlabel("days")

- plt.xlabel used to label the x axis in graph
- here days is the string to show as x axis

4] plt.ylabel("temperature")

- plt.ylabel used to label the y axis in graph.
- Here temperature is the string to show as y axis.

5] plt.title("Delhi Temperature")

- plt.title() used to label the graph.

6] plt.plot('days', delhi_temp, "go--", linewidth=3, markersize=10)

- g is green value of color
- o is circle of marker
- -- is the line style
- linewidth is the width of line
- markersize decide the size of marker/points

7] plt.plot (label = "Temp line")
plt.legend ()
- label create a short label in corners
- plt.legend () shows that label
- you can change location of label by values 0 to 10

8] from matplotlib import style
style.use("ggplot")
plt.grid (color="r", linestyle="-", linewidth=2)
- If you want to change style of graph.
- You need to import style
- style.use ("ggplot") is one type of style.
- plt.grid is the function for grids in graph
- You can change color, style, width.

9] Add more than one point/marker in graph
- In given example delhi_temp is already added in it
- You can add more cities temperature in it.
- Use plt.plot (parameters) for that values.

Histogram

1] `plt.hist(List_1)`
- Create histogram means bar type graph.

2] `plt.show()`
- Use this function to show graph

3] `plt.hist(List_1, bins)`
- bins is used for ~~bar of axis~~ values like [0,5,10,15,20,25]

4] `plt.hist(List_1, rwidth=0.8)`
- rwidth create a distance between bars

5] `plt.hist(List_1, orientation='horizontal')`
- bars views changes by horizontal orientation

6] `plt.hist(List_1, color='m')`
- Color changes the color of bars by giving value.

7] `plt.hist([List1, List2])`
- You create more than one bar values using list

8] `plt.figure(figsize=(10,9))`
- Function used for graph size

Bar Chart

1] `plt.bar(classes, class1_students)`
- Create bar chart with classes at x-axis & class1_students at y-axis

2] `plt.barh(classes, class1_students)`
- Create a bar chart horizontally.

3] `plt.bar(classes, class1_students, width=0.2)`
- width of bar is 0.2

4] `plt.bar(classes, class1_students, align="edge")`
- Create edges for bar

5] `plt.bar(edgecolor="r")`
- Color of edges of bar.

6] `plt.bar(linewidth=5)`
- width of edges of bar

7] `plt.bar(linestyle="--")`
- style of edges of bar

8] `plt.bar(visible=False)`
- Hide bars

width = 0.2

9] class_index = np.arange(len(classes)),
plt.bar (classes_index, class1)
plt.bar (classes_index + width, class2)
• plot two bars for class1 & class2
• x-axis is class_index

width = 0.2

10] class_index = np.arange(len(classes))
plt.bar (classes_index, class1)
plt.bar (classes_index + width, class2)
plt.bar (classes_index + width + width, class3)
• plot three bars for class1, class2, & class3.
• width is 0.2 & used for plotting.

* Scatter Plot

1] plt.scatter(x,y)
- Create scatter graph using x & y.

2] plt.scatter(x,y, c="m")
- c means color of scatter

3] plt.figure(figsize=(10,9))
- Changes size of figure/graph to 10 x 9

4] plt.scatter(x,y, c="m", marker="*")
- marker changes shape of scatter

5] plt.scatter(x,y, s=100)
- s is the size of scatter

6] plt.scatter(x,y, alpha=0.5)
- alpha decides the transperency of scatter

7] plt.scatter(x,y, linewidth=10)
- linewidth changes line width of scatter

8] plt.scatter(x,y, edgecolor="g")
- edgecolor changes color of edge of scatter

9] plt.scatter(x,y)
plt.scatter(x,z)
- Plots more than one scatter in graph.

PIE CHART

1] plt.pie(class1_students)
- class1_students is a list with 5 values
- plt.pie creates a pie charts.

2] plt.pie(class, labels = classes)
- label parameter labels that partitions
- classes is the list with 5 strings.

3] explode = [0.3, 0, 0.1, 0, 0]
plt.pie(explode=explode)
- explode is a list with 5 values
- explode parameter uses that values & create slices of partition & move from pie with given value.

4] plt.pie(colors = colors)
- colors is a list with 5 color values that can changes the partition color.

5] plt.pie(autopct = "%0.2f%%")
- autopct means auto percentage. 0.2 means 2 value after point.

6] plt.pie(shadow=True, radius = 1.4, startangle=90)
- shadow if True shows shadow in pie.
- radius decides pie size or circle size.
- startangle used for starting line of partition.

* SubPlot

1] plt.subplot(...)
- This function plot more than you want graphs
- By using this function you can plot any type of graph. multiple with diff. graph in one figure.

2] plt.subplot(3,2,1)
- 3 is the value of row
- 2 is the value of column
- 1 means this is first graph of subplot.
- Here, 3 rows & 2 columns means you can plot 6 graphs.

3] plt.subplot(321)
- Without using commas also you can plot graphs.

4] plt.subplot(3,2,3)
plt.hist(---)
- If you want to plot the diff. graph code it under subplot line.

5] plt.subplot(3,2,6, projection='polar', facecolor='k')
- Plot polar projection in graph.
- facecolor is the background color of Axes

savefig

- plt.savefig("path_or_filename", facecolor="k").
- You can save the graph or figure in png format in your directory using savefig function
- You can give path for location or just filename
- facecolor is the background of graph.

Image function.

```
import matplotlib.image as mpimg
```

- img = mpimg.imread("pie-char.png")
- used for read the image
- But img will be in array of numpy format.

- plt.imshow(img)
- Shows real image

- plt.colorbar()
- Create color bar in fig.

- single_channel = img[:,:,1]
plt.imshow(single_channel, cmap="hot")
- cmap = "hot" & also have other values.
- img[:,:,1] value are need to use for better view.

* SEABORN *

* What is seaborn ?
  - Seaborn is a python library for data visualization build on top of the matplotlib Python library
  - Use for statistical graphics.

* Seaborn Graphics.
  - Heatmap
  - Pairplot
  - Facet grid
  - Bar Plot
  - ScatterPlot
  - Line Plot
  - Dist Plot
  - Box Plot
  - Violin Plot

* Dependency of Seaborn
  - Python
  - Numpy
  - Pandas
  - Scipy
  - Matplotlib

* LINE PLOT *

1] sns.lineplot(x="day", y="temp", data=temp-df)
- temp-df is a data frame.
- day is on the x axis
- temp is on the y axis.

*2] sns.load-dataset("tips")
- You can take dataset from Github. without download it.

3] sns.load-datasetlineplot(hue="sex")
- hue shows specific folders data

4] sns.linplot(style="sex")
- style for sex for column data

5] sns.lineplot(palette="Accent")
- You can change diff. colors or style of lineplot using palette.
- Accent is given value, you can give another value

6] sns.lineplot(marker=["o", "<", ">", "^"]
- You need to give the no. of marker for every category in given column of hue = "day".

BARPLOT

1] sns.barplot(x = tips_df.day, y = tips_df.total_bill) or
sns.barplot(x = "day", y = "total_bill", data = "tips_df")

- You can plot a barplot using above two type.
- In first syntax directly use the column data by Df name.
- In second syntax directly use the column name & then write Df in data.

2] sns.barplot(hue = "sex")

- hue customize data into given column's data.

3] order = ["Sun", "Fri", "Thur", "Sat"]
sns.barplot(order = order)

- You can change the serial of x axis using order.

4] hue_order = ["Female", "Male"]
sns.barplot(hue_order = hue_order)

- You can change the order of hue column's data by using hue_order

* SCATTERPLOT *

1] sns.scatterplot(x="age", y="fare", data = titanic-df)
- You can plot a scatterplot using above function
- data is mandatory if you pass only column names to x & y.

2] sns.scatterplot(hue="sex")
- hue is used to customize data in given column.

3] sns.scatterplot(style="who")
- style also used to customize data in given column but it changes ~~the~~ style of scatter for diff. data.

4] sns.scatterplot(size="who", sizes=(100, 400))
- size changes the size of scatter of given column names data in given sizes.

* Histplot *

- sns.histplot(series, kde=True)
- Used for frequency

* Boxplot *

- sns.boxplot()